The email_downcased column will use the SQL LOWER() function to modify the input. The create_table snippet for the migration is shown below.

Generate a new Rails Migration and add the following implementation to the change method.

ruby/migration_snippet_virtual_column.rb

```
create_table 'temp.customers' do |t|
  t.string :email
  t.virtual :email_downcased,
    type: :string,
    as: 'LOWER(email)',
    stored: true
end
```

Run bin/rails db:migrate again to apply it. Launch psql and run \d temp.customers to view both columns.

The email_downcased will now have the following Default configuration:

```
GENERATED ALWAYS AS (LOWER(email::text)) STORED
```

The GENERATED keyword refers to the Generated Column. ALWAYS AS ... STORED shows that the transformed value is automatically stored that way. The LOWER() function is used as an expression to make the column data lowercase.

A benefit of the Generated Column approach is that it does not require enabling the citext extension.

On the other hand, it does require adding a another column to your schema. Consider whether you'd rather have an extension or the schema change.

In the next section you'll continue exploring consistency and correctness techniques you can add to your database.

PostgreSQL supports Enumerated Types that can be added directly to your database. If you're a seasoned Rails developer, you may be familiar with Enums at the application code layer but perhaps not in the database.

It's time to put those into action.

Constraining Values With Database Enums

Enumerated Types or *Enums* are a way to express a limited explicit list of possible values. These are the full comprehensive set of possible values that can exist. In a way, they are similar to Constraints in that they *constrain* the possible values.